

Original article

Large language model agent for user-friendly chemical process simulations

Jingkang Liang¹, Niklas Groll^{1,*}, Gürkan Sin¹*Process and System Engineering Center, Department of Chemical and Biochemical Engineering, Technical University of Denmark, Søtofts Pl. 228A, 2800, Kgs. Lyngby, Denmark*

ARTICLE INFO

Dataset link: <https://github.com/gsi-lab/APS-Agent>

Keywords:

Chemical process simulation
Large language model
Model Context Protocol

ABSTRACT

Modern process simulators enable detailed process design, simulation, and optimization; however, constructing and interpreting simulations is time-consuming and requires expert knowledge. This limits early exploration by inexperienced users. To address this, a large language model (LLM) agent is integrated with AVEVA Process Simulation (APS) via Model Context Protocol (MCP), allowing natural language interaction with rigorous process simulations. An MCP server toolset enables the LLM to communicate programmatically with APS using Python, allowing it to execute complex simulation tasks from plain-language instructions. Three case studies assess the framework across different task complexities and interaction modes. The first shows the agent autonomously analyzing flowsheets, finding improvement opportunities, and iteratively optimizing, extracting data, and presenting results clearly. The framework benefits both novice users (for educational purposes), by translating technical concepts and demonstrating workflows, and experienced practitioners, by automating data extraction, speeding routine tasks, and supporting brainstorming. The next case study assesses autonomous flowsheet synthesis through both a step-by-step dialogue and a single prompt, demonstrating its potential for novices and experts alike. The step-by-step mode gives reliable, guided construction suitable for educational contexts; the single-prompt mode constructs fast baseline flowsheets for later refinement. Further, the agent's answers proved stable across variations in LLM versions, prompting types, and process flowsheet complexities. While current limitations mainly involve the accurate interpretation of the physical results seen in minor reasoning mistakes, such as oversimplification or misleading suggestions, mean expert oversight is still needed. However, the framework's capabilities in analysis, optimization, and guided construction suggest LLM-based agents can become valuable collaborators for process simulation tasks in future.

1. Introduction

Chemical process simulators serve as an indispensable cornerstone of modern chemical engineering practice, supporting engineers to design, analyze, and optimize complex industrial systems. From petrochemical refineries to pharmaceutical manufacturing plants (Suzuki et al., 2012), these sophisticated computational tools enable engineers to predict process behavior, evaluate design alternatives, and troubleshoot operational challenges before costly physical implementation (AVEVA Group plc, 2025; AspenTech, 2025). However, despite decades of technological advancement, traditional simulation workflows remain plagued by significant limitations that hinder their widespread adoption and effectiveness.

The current state of process simulation is characterized by steep learning curves, extensive manual configuration requirements, and the need for deep domain expertise that can take years to develop (Y.

Huang et al., 2025). Engineers must navigate complex software interfaces, manually select thermodynamic property models and operating parameters, and possess intimate knowledge of underlying physical principles to achieve meaningful results. This complexity creates substantial barriers, particularly for junior engineers, interdisciplinary teams, and organizations with limited simulation expertise, effectively restricting the accessibility of these powerful mechanistic tools (Dahm et al., 2002).

Simultaneously, the artificial intelligence (AI) landscape has witnessed unprecedented breakthroughs with the advent of Large Language Models (LLMs) and sophisticated AI agent systems (Ferrag et al., 2025). These technologies have already demonstrated remarkable success across diverse domains, from autonomous software development and scientific research to complex problem-solving tasks that require intricate tool integration and coordination (Chu et al., 2025; Dong et al.,

* Corresponding author.

E-mail address: nigro@kt.dtu.dk (N. Groll).¹ Jingkang Liang and Niklas Groll contributed equally to this work and share first authorship.

2025; Yang et al., 2025; Xiao et al., 2024). The emergence of agentic AI systems, capable of planning, reasoning, and executing multi-step workflows, presents a compelling opportunity to address the accessibility challenges of conventional chemical process simulators (Rupprecht et al., 2026).

This combination of mature simulation technologies with cutting-edge AI capabilities raises fundamental questions about the future of chemical engineering practice: Can the sophisticated reasoning abilities of LLM-based agents be effectively harnessed to assist (inexperienced) users in navigating complex simulation workflows? How can we maintain the deterministic accuracy and safety standards required in chemical engineering while leveraging the accessibility benefits of conversational AI interfaces?

Addressing these questions, this work presents a novel framework that strategically couples an LLM-based agent with a commercial chemical process simulation software via a custom toolset. Rather than replacing traditional simulation methodologies and workflows, our approach enhances them by providing an intelligent, conversational interface that should maintain the underlying rigor and reliability of commercial process simulators. The proposed framework is designed to serve as an assistant to experienced users in their daily tasks using the conventional process simulator while simultaneously guiding new users through complex tasks of analyzing and synthesizing process simulation flowsheets, thereby broadening access to advanced simulation capabilities without compromising engineering standards. The novelty of this work lies in a structured, modular integration of an LLM with a commercial process simulator through standardized protocols, combining a curated, extensible tool interface with a trustworthiness-oriented evaluation across guided and higher-autonomy operation.

The following sections of this paper are organized as follows: Section 2 reviews the current state of the art in LLMs, agentic AI systems, and their applications in engineering domains, in particular, chemical engineering. We highlight existing gaps and challenges and formulate this work's aim. Section 3 details the proposed agentic framework, including the selection of an LLM agent, its integration with an agentic protocol, and its integration with the commercial simulation software via a custom toolset. Section 4 presents three case studies testing the practical applications of the framework in process analysis and synthesis and its stability, before Section 5 discusses the results and implications of these case studies. Finally, Section 6 concludes with a summary of key findings, implications for chemical engineering practice, and directions for future research.

2. State of the art

The development of LLMs has undergone a remarkable evolution, beginning with the invention of the foundational transformer architectures by Vaswani et al. (2017) in 2017 and culminating in sophisticated reasoning systems capable of complex problem-solving (Wei et al., 2022a). Early language models primarily focused on text generation and completion problems, but the introduction of instruction tuning (Zhang et al., 2026), reinforcement learning from human feedback (Bai et al., 2022), and chain-of-thought reasoning (Wei et al., 2022b) has dramatically expanded their capabilities beyond simple text generation to sophisticated reasoning and problem-solving tasks. The prevailing consensus among researchers and practitioners suggests that we are witnessing the early stages of the LLM era (Matarazzo and Torlone, 2025), with ongoing improvements and breakthrough discoveries anticipated in the coming years (Zhang et al., 2025), particularly given the substantial computational resources and capital investments being directed toward this field.

The latest step, featuring agentic AI systems, represents a fundamental paradigm shift from monolithic language models to autonomous reasoning entities capable of planning, executing, and reflecting on complex multi-step workflows, a critical advancement for achieving universal applicability of LLM-based solutions (Wang et al., 2024).

These AI agents leverage LLMs as cognitive cores while integrating external tools, APIs, and environments to perform specialized tasks that extend far beyond natural language processing (Derouiche et al., 2025). Contemporary agent architectures employ sophisticated planning algorithms, memory management systems, and tool orchestration capabilities that enable them to decompose complex objectives into executable subtasks, monitor progress, and adapt strategies based on intermediate outcomes (Yao et al., 2023; Schick et al., 2023).

To govern the operational integration of these autonomous entities, recently standardized agentic protocols have been developed to define formal rules for system interoperability. A primary example is the Model Context Protocol (MCP), launched by Anthropic in 2024 (Anthropic, 2025b). MCP provides a universal, vendor-neutral standard for connecting an agent's cognitive core to external toolsets and data environments (Anthropic, 2025b). Unlike traditional custom integrations, MCP enables a decoupled architecture where agents dynamically discover and invoke capabilities through a unified interface. This standardization supports both single-agent systems, which provide efficient, centralized reasoning for direct task execution, and Multi-Agent Systems (MASs), which distribute objectives across specialized entities to handle high-dimensional problems through parallelized expertise. Single-agent architectures often remain preferable for tasks that require lower latency and reduced computational overhead, while MAS can offer greater robustness through peer-review loops (Du et al., 2024). Here, orchestration frameworks can play a crucial role in these configurations. Frameworks such as LangGraph (LangChain Inc., 2025) and AutoGen (Microsoft, 2025) provide infrastructure for cyclic, stateful workflows, while Cognition (Cognition AI Inc., 2025) demonstrates specialized autonomous engineering through integrated planning and environmental interaction. Additionally, high-level protocols such as Agent2Agent (Google Developers, 2025) can help coordinate orchestration between these systems, providing an overarching structure that ensures seamless integration and cooperation between diverse agentic components. Together, these developments facilitate practical applications of LLMs across technical domains, including tasks such as education, software development, scientific research, and data analysis.

Frequently, LLM systems are developed first for educational purposes; LLM-driven teaching assistants have been deployed to generate formative feedback, design learning materials, and personalize instruction for students, ultimately enhancing learning experiences and saving educators time (Chu et al., 2025). In software development, agentic coding systems can autonomously plan tasks, write code, execute tests, analyze failures, and iteratively debug programs until they succeed, transforming developers' roles by reducing the mundane aspects of coding and allowing them to focus on more innovative tasks (Dong et al., 2025). In scientific research, LLM agents support multi-stage workflows by interpreting high-level research goals, retrieving literature or data, designing experiments, and coordinating external computational tools to implement partially automated research workflows, reducing researchers' workload and expediting discovery (Ferrag et al., 2025). In drug discovery, MASs combine reasoning, retrieval, and search algorithms to propose candidate molecules, evaluate properties, and refine designs across iterative discovery loops, aiding scientists in accelerating the drug development process and increasing efficiency (Yang et al., 2025; Li et al., 2025). Collectively, these applications demonstrate that LLM-agents are no longer limited to text generation, but have evolved into autonomous, tool-using systems capable of structured reasoning, planning, and multi-step execution across a wide range of professional domains.

Also across the field of chemical engineering, efforts have been made to leverage the power of LLMs and build agentic AI systems that support core activities. Many integrations relate to chemical engineering education. Tsai et al. (2023) highlight the potential benefits of integrating LLMs like ChatGPT as a problem-solving tool to enhance students' critical thinking; Caccavale et al. (2025) introduced ChatGMP, an LLM-based chatbot that supports student question answering by

retrieving and synthesizing information from lecturer audio recordings using a retrieval-augmented generation framework. Many other studies similarly report that LLMs can support lab activities (Keith et al., 2025), virtual tutoring (Caccavale et al., 2024), and process-design learning (Kong et al., 2023), reinforcing their growing value as educational tools. Further, models such as Coscientist (Boiko et al., 2023), ChemCrow (M. Bran et al., 2024), ChemLLM (Zhang et al., 2024), and BatGPT-Chem (Yang et al., 2024) enable autonomous experiment design, laboratory automation, retrosynthesis planning, and multi-tool chemical reasoning, highlighting how LLM agents bridge natural-language intent with computational chemistry workflows. In fault diagnosis, systems like FaultExplainer (Khan et al., 2025) leverage LLM reasoning to interpret detected faults, explain sensor contributions, and propose plausible root-cause hypotheses, thereby improving operator interpretability. Finally, multiple studies explore LLM applications for chemical process design tasks, such as process analysis and synthesis, including process simulations. Du and Yang (2025) outline a next-generation agent architecture for intelligent simulation workflows; Lee et al. (2024) introduce a multi-agent GPT-based system that generates process design improvements from structured flowsheet descriptions; and Zeng et al. (2025) demonstrate an LLM-guided multi-agent optimization system that autonomously infers operating constraints and outperforms traditional methods on hydrodealkylation optimization. In simulation automation, Liu et al. (2025) show that autonomous simulation agents can plan, execute, and analyze full research simulations, achieving multi-cycle long-task execution with minimal human input, while Rajeev et al. (2025) integrate an AI assistant directly with the commercial platform AVEVA Process Simulation (APS) to support engineers in developing and troubleshooting industrial flowsheets. But in contrast to our work, they are fine-tuning local LLMs to directly modify the simulation written in extensible markup language (XML) and are not using state-of-the-art LLM systems and the provided APS Python interface, resulting in a tightly coupled solution that is harder to transfer, extend, and systematically evaluate across different workflows. To the best of our knowledge, no prior study reports a fully implemented, reproducible MCP-based integration with a commercial process simulator that is evaluated in both guided and higher-autonomy (dual-mode) operation. Nevertheless, these advancements collectively indicate that LLM-based technologies have the potential to transform multiple facets of the chemical engineering lifecycle, including classroom instruction, research, industrial-scale simulation, and process design.

Despite recent advancements, key challenges remain in developing trustworthy LLM-based assistants that can truly support experienced chemical engineers in complex, real-world tasks. Specifically, existing LLM-based technologies have not yet addressed tasks requiring deep system understanding and decomposition into interconnected sub-problems, such as the analysis and synthesis of industrial production systems (Seider et al., 2016). Chemical engineers rely on specialized simulation software for rigorous solutions, but these tools often pose usability challenges due to manual configuration requirements and the need for substantial domain expertise. Thus, a fundamental gap exists in transforming traditional, human-centric simulation software into flexible, conversational tools powered by LLMs, while preserving the rigor and reliability necessary for safety-critical engineering applications. More specifically, the open gap is to achieve scalability for long, multi-step tasks, modularity between reasoning and simulator execution, and transparent trustworthiness assessment under different autonomy levels.

To address this gap, our work aims to develop and evaluate an LLM agent framework, built on MCP, that enables natural-language interactions with the commercial simulator APS. Using MCP as an explicit abstraction boundary between the LLM and simulator-specific operations, this framework is designed to ensure safe, reliable, and reproducible execution by integrating a curated toolset, exposed through the APS Python API, that bridges conversational AI with deterministic

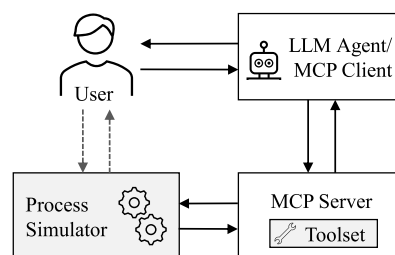


Fig. 1. Schematic overview of the proposed framework with MCP.

process simulation and enables true agentic autonomy (planning, iterative tool orchestration, and multi-step execution). We demonstrate the framework's practical utility through two comprehensive case studies, showcasing its application in process understanding, analysis, optimization, and flowsheet synthesis. Finally, we critically assess the strengths and limitations of LLM-driven agents in supporting chemical engineering workflows across both guided and higher-autonomy interaction modes, highlighting how such systems can make commercial process simulators more accessible and efficient without sacrificing the rigor required for industrial use.

3. Methodology

Our proposed framework, as illustrated in Fig. 1, intends to replace the direct, manual interaction between a user and a commercial process simulator with an intelligent, conversational interface powered by an LLM. Instead of navigating the simulator's graphical user interface or writing custom scripts, the user communicates with an MCP-enabled LLM agent, also called the MCP client, in natural language. The LLM agent interprets the request, plans the necessary actions, and invokes a curated toolset exposed by a dedicated MCP server. The server, in turn, interfaces with the process simulator and executes all deterministic engineering operations. This architecture ensures that the LLM only performs high-level reasoning and planning while all physics-based computations remain under the control of the commercial simulator's validated solver. This section details the framework's architecture, covering the MCP client, the MCP server, and the integration with the process simulator, including the curated toolset.

3.1. LLM agent and MCP client

The LLM agent, running within a host application, serves as the primary interface to the simulation environment. It interprets natural-language requests, decomposes complex tasks, orchestrates tool calls, and maintains conversational context. This contextual reasoning is essential for ensuring that the system correctly interprets user intent and provides accurate, relevant responses. Additionally, the LLM agent should detect and handle user errors and present results in clear, user-friendly natural-language outputs.

Selecting a suitable host application, LLM agent, and underlying model involves careful consideration of several criteria, including strong reasoning capabilities for multi-step tool planning, reliable tool-use behavior, and a sufficiently large context window to preserve conversation history. For this work, we chose Claude Desktop, powered by Claude Sonnet 4.0 (Anthropic), as our default LLM agent (Anthropic, 2025a). This decision was motivated by its state-of-the-art performance on reasoning tasks and tool-use benchmarks in technical and scientific domains (LMarena Team, 2025).

Within this architecture, Claude Desktop acts as the host application, instantiating an MCP client responsible for protocol-level communication. The LLM interacts with this client, instructing it to discover, connect to, and invoke tools exposed by MCP servers. Because Claude Desktop includes native MCP client functionality, this setup enables seamless integration with custom MCP servers without requiring additional middleware or custom communication layers.

3.2. MCP server

The MCP server acts as the intermediary layer between the LLM agent and the process simulation software. By abstracting the simulator's Python API into a clean, high-level toolset with clearly defined inputs, outputs, and usage descriptions, the MCP server enables the LLM to interact with the simulator without requiring model-specific customization. Because the LLM only accesses these abstract tool definitions, rather than the simulator's internal API, the same toolset can be reused across different LLMs or host applications, enabling seamless migration and future extension of the framework. This decoupling keeps the integration aligned with rapid improvements in underlying LLM capabilities, since model upgrades can be adopted without redesigning the simulator interface layer.

Our implementation uses FastMCP, a modern Python framework for building lightweight MCP servers (Lowin, 2025). FastMCP provides a streamlined interface for defining tools, specifying request and response schemas, and handling communication via standardized JSON-RPC messaging. Each tool is implemented as a typed Python function annotated with metadata describing its parameters, return types, and purpose. This design pattern simplifies server development while providing automatic schema generation and robust tool discovery for the host application.

The MCP server runs as a standalone process, and the host application (Claude Desktop) connects to it through its built-in MCP client. Once connected, the LLM agent can query available tools, invoke them with appropriate arguments, and receive structured responses containing simulation data or execution results. Although the underlying LLM is general-purpose, this MCP tool layer functions as a domain-specific operational schema: the model can only invoke typed tools with predefined arguments and receives structured outputs. This constrains the interaction space, reduces ambiguity, and improves consistency without domain-specific model retraining. This architecture ensures a reliable, transparent communication pipeline, enabling the LLM agent to coordinate simulation steps while deterministic computations are handled by the server and the underlying simulator.

3.3. Process simulator integration

The connection between the MCP server and the employed process simulator, APS (version 2025), is established via the APS scripting interface, a comprehensive Python-based API that provides programmatic access to all core simulation functionalities, including flowsheet construction, parameter manipulation, and result extraction. Our custom MCP server implementation wraps selected APS scripting interface functions into a curated set of high-level tools. Each MCP tool encapsulates one or more APS API calls, which are the building blocks of the tools, into a single meaningful operation, aligned with common simulation steps a process engineer would take.

The full toolset exposed by the MCP server is presented in Supplementary Material S1. In total, it comprises 16 tools and enables the MCP client to perform a wide range of simulation tasks, from simple data queries and flowsheet inspections to the construction of new flowsheets by adding models and connectors. The tools can be grouped into key categories covering connecting to and opening simulations, saving and checking simulation status, retrieving flowsheet topology, getting and setting variables and parameters, adding and connecting models, and creating and using fluids for thermodynamic setup. Importantly, individual tools may wrap multiple original APS API calls when appropriate and transform the original returns into structured JSON files, which are returned to the MCP client. An example is the tool `var_get_multiple`, which can call the function `Get_Variable_Value` multiple times within a single coordinated operation and returns the results for multiple variables in a single structured JSON file. The modular design of the MCP server architecture ensures the toolset is easily extensible when additional functionality is needed, enabling future expansion

with capabilities such as automated sensitivity analysis, optimization routines, or advanced convergence troubleshooting without requiring changes to the underlying LLM agent configuration. Critically, the MCP server serves as a semantic bridge, orchestrating high-level simulation logic through natural language reasoning and tool selection. It translates conversational intent into rigorous engineering actions, maintaining complete control over the deterministic execution logic. At the same time, all actual computations remain firmly grounded in the deterministic, physics-based solver algorithms provided by APS.

3.4. Evaluation

To evaluate the agents' answers, we employ both qualitative and quantitative criteria to assess correctness, completeness, efficiency, and user satisfaction, which directly relate to safety, reliability, and effectiveness of the framework.

The quantitative assessment focuses on objective measures of performance, including the number of (correct) tool calls, the number of parameters and variables read or set, the number of numerical values displayed to the user, and the frequency with which information is factually wrong or numerically inconsistent. In addition, we list the total number of technical errors, such as mistakes in tool calls or values displayed, as well as qualitative errors in the LLM agent's explanations. These measures operationalize reliability primarily through protocol fidelity (correct tool names, arguments, and variable paths) and error frequency, and they provide a proxy for effectiveness by indicating whether the agent can execute the required sequence of simulator interactions to obtain the requested results with limited iteration and rework. Safety-relevant failures are captured indirectly as technical errors and invalid actions rejected by APS (e.g., inconsistent specifications or non-converging solver states), indicating that the agent attempted operations that violated simulator constraints.

In complement to the quantitative evaluation, the qualitative assessment captures additional aspects not well represented by raw numbers. Here, we focus on dimensions such as safety, ensured by physical consistency and adherence to APS-enforced guardrails (mass and energy balances, thermodynamic consistency, and simulation-state validation). In terms of effectiveness, we assess whether the answer addresses the engineering objective and whether the resulting explanation is clear, structured, and actionable for decision-making. Finally, qualitative analysis also supports reliability by evaluating whether the narrative remains consistent with the tool outputs and whether limitations and uncertainty are communicated transparently. This broader evaluation is important because a technically correct response can still be difficult to use if it is poorly organized, overly verbose, or insufficiently explicit about the engineering implications of the results. To reduce misinterpretation risk, we combine curated MCP tools, APS validation, and human-in-the-loop review; accordingly, recommendations are treated as hypotheses that require verification before implementation.

4. Case study applications

To demonstrate the practical application of our proposed agentic framework, we select two existing simulation models from the APS examples library as a baseline and comparison flowsheet. The simulation example "C1 - Water Methanol Separation" models the binary separation of methanol and water via distillation, and is detailed in the next section. Based on this example, we design two case studies to test our proposed workflow. The case studies target common tasks of process simulation engineers, the (1) analysis of simulation flowsheets and (2) synthesis of simulation flowsheets detailed in Sections 4.2 and 4.3. Finally, we employ the simulation example "A6G - Ammonia Synthesis EQ" during the stability assessment in Section 4.4.

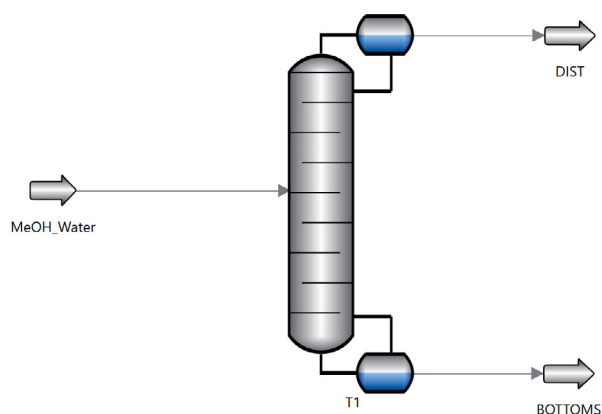


Fig. 2. Simulation flowsheet of APS example “C1 - Water Methanol Separation”.

4.1. Water methanol separation process

Methanol serves as a key feedstock for the production of various chemicals and fuels making its production and purification critical processes in the chemical industry (Bozzano and Manenti, 2016). In particular, the mixture of water and methanol is commonly encountered in industrial processes, for example in biodiesel production, where methanol is used as a reactant and must be first recovered from the aqueous phase to facilitate the downstream processing steps (Burmana et al., 2025).

For analyzing the process configuration, an understanding of the fundamental chemical and physical principles is necessary. Although the boiling points of methanol (64.7 °C) and water (100 °C) at ambient pressure differ by approximately 35 °C, suggesting a relatively straightforward separation, this binary system features some interesting challenges. The water-methanol mixture exhibits non-ideal vapor-liquid behavior, characterized by positive deviations from Raoult’s law, requiring accurate thermodynamic models, such as NRTL, to capture phase equilibrium correctly (Kurihara et al., 1993). While the system does not form an azeotrope, the relative volatility varies significantly with composition, affecting separation efficiency across the column height. Due to these non-ideal behaviors, careful consideration of design (e.g., number of stages) and operational (e.g., reflux ratio) parameters is required to achieve desired product purities while maintaining energy efficiency. These characteristics position the binary water-methanol separation problem as an ideal benchmark system for process analysis and optimization. The system is simple enough for rapid analysis by domain experts, yet sophisticated enough to test the LLM agent’s understanding of thermodynamic principles, equipment configuration, energy-efficiency considerations, and process-optimization strategies in a realistic industrial context.

The process simulation flowsheet for the water-methanol separation, shown in Fig. 2, is rather simple, consisting of a source stream, a distillation column equipped with both a reboiler and a condenser, and sink modules for collecting the bottom and distillate products. We define the feed stream as an equimolar mixture of water and methanol, with a flow rate of 10 kg/s, at the specified operating conditions (e.g., 80 °C and 1 atm). Critical parameters that can be adjusted to optimize the separation include reflux and reboiler ratios, the number of stages in the distillation column, the feed stage location, and overall operating pressure and temperature profiles. While the synthesis of the process requires careful selection of appropriate modules, considering the underlying chemical and physical principles to obtain a converged

simulation flowsheet, the number of steps required is limited, implying a fast flowsheet construction for experienced users. Therefore, the binary water-methanol system provides a realistic yet manageable example that captures the key challenges of synthesizing industrial separation processes.

4.2. Case study 1: Analysis of the simulation flowsheet

In the first case study, we aim to evaluate the LLM agent’s ability to analyze, interpret, and improve the existing water-methanol separation simulation modeled in APS. The task is structured into two distinct steps. First, we ask for a detailed summary of the modeled process, and second, we request a list of potential process improvements. This case study should assess the system’s capacity to comprehend complex process simulation models, communicate findings effectively to the user, and offer expert-aligned recommendations for system improvements.

The initial user prompt asks for a detailed summary of the specified simulation file. To fulfill this request, we expect the agent to connect to APS, open the relevant simulation file, and gather data on all present modules and their connections. Further, it should independently identify and extract the values of relevant key process variables and parameters. After screening all extracted information, we expect to obtain a structured, detailed natural-language description of the simulated process. The summary should include the flowsheet structure and key process specifications, such as feed composition, operating conditions, and energy demands.

In the second step, the user asks the agent how to improve the simulated process. Based on the previously extracted information, the agent can decide to extract more information from the simulation model if needed. After that, the agent should provide a summary of multiple points that could improve the separation task based on different metrics, such as separation efficiency, purity of the product streams, or energy demand. Dependencies between different variables and operating conditions should be clearly stated. Finally, a comprehensive list of suggested improvements should be provided to the user.

4.3. Case study 2: Synthesis of the simulation flowsheet

The second case study assesses the agent’s process synthesis capability with the primary goal of constructing an entire simulation flowsheet in APS from high-level user specifications. We evaluate and compare two construction modes based on different user interactions and prompting: first, we test the assembly with a stepwise dialogue between the user and the LLM agent; second, we provide the agent with a single user prompt.

In the first approach, the user has almost full control, guiding the agent to assemble the simulation flowsheet by incrementally requesting the addition of unit operations, connection of units, and setting of parameter values. This approach mirrors manual construction, but instead of manually clicking through the APS graphical user interface, the user now provides the LLM agent with the required next step. This way, the user can control that each step is conducted correctly and provide help, or even take over a step, when the agent encounters problems. The user reduces the complexity of each step by breaking the task into steps so that the agent requires only a single tool call. Concurrently, the complexity can also be enhanced by combining subtasks into the next user prompt. In this mode, we test the agent’s ability to follow stepwise user instructions to construct the simulation flowsheet in APS using the provided toolset.

In contrast, with the second mode, we evaluate the agent’s capacity to autonomously construct a simulation flowsheet from a single user prompt. This mode requires the agent to possess a sophisticated, abstract understanding of the problem statement and to independently decompose the task into executable subtasks using the available toolset. The process requires orchestrating multiple sequential tool calls with

progressively increasing information availability to fulfill this complex objective. With this mode, we test the agent's ability to independently plan and execute a multi-step workflow to synthesize a complete process simulation flowsheet based on concise high-level user specifications.

4.4. Case study 3: Stability assessment

Finally, we perform additional stability experiments to evaluate the robustness and reliability of the framework in greater detail. To assess stability, we vary the framework and task setup relative to the default trial (Prompt 1.0 in Case Study 1) by changing the LLM version, prompting style, and process complexity. The setup of all trials is detailed in Supplementary Material 3.3.

5. Results and discussion

In the following, we present and discuss the results obtained from the three case studies detailed before. The performance of the LLM-based agent in analyzing and synthesizing simulation flowsheets in APS is quantitatively and qualitatively evaluated. While we show only excerpts in this section, all full prompts and answers can be found in the Supplementary Material S2, and more detailed evaluations including all quantitative metrics of the answers are listed in S3.

5.1. Case study 1: Analysis of the simulation flowsheet

In the first case study, we evaluate the agent's ability to analyze the water-methanol separation simulation modeled in APS. We initiate the workflow by prompting the agent with a very general prompt that can also be reused for analyzing any other simulation flowsheet in APS. The prompt provides no specific guidance on how to analyze the flowsheet or what information to extract but we place importance on having a well-structured prompt by specifying the agent's role, task, the required steps, and the desired output format.

Prompt 1.0

Role: You are a process simulation expert specializing in chemical process modeling using AVEVA Process Simulation (APS). You can use the predefined APS toolset to connect, open, modify, inspect, and analyze simulations.

Task: Analyze the flowsheet named "C1 - Water Methanol Separation"

Required Steps:

- Connect to APS.
- Open the specified simulation file.
- Retrieve the flowsheet topology, including all models and connectors.
- Read and report key variables and parameters from relevant models (equipment, streams, controllers, etc.).
- Produce a consolidated summary of all extracted information.

Return Format: Provide a short bullet-point summary (maximum 500 words)

- Major equipment and stream connectivity
- Important operating conditions or parameters
- Any noteworthy observations from the simulation data

Answer 1.0

I will analyze the "C1 - Water Methanol Separation" [...] start by connecting to APS and opening the simulation.

→ `aps_connect()`

Great! [...] Now let me retrieve the flowsheet topology to understand the models and connectors.

→ `models_list()`

→ `connectors_list()`

Now let me examine the key variables and parameters from the main equipment (column T1) and the feed source.

→ `model_all_vars(T1)`

→ `model_all_vars(MeOH_Water)`

Now let me get the key stream variables to understand the flows and compositions:

→ `var_get_multiple(S1.F, S1.W, ...)`

Let me also check the simulation status to see if it's converged:

→ `sim_status()`

Analysis Summary: C1 - Water Methanol Separation Flowsheet

Major Equipment and Stream Connectivity:

- Column T1: 13-stage distillation column [...]

Important Operating Conditions: [...]

Key Performance Data: [...]

Noteworthy Observations:

- The simulation is fully converged and properly specified
- Feed enters at stage 7 (middle of 13-stage column), optimal for this separation
- Very low methanol concentration in bottoms (0.002 mol%) indicates excellent separation
- Energy balance is satisfied [...]
- Standard tray efficiency of 100% assumed throughout the column

The agent's response to Prompt 1.0 demonstrates a systematic, methodical approach to analyzing the simulation flowsheet in APS. Relevant information is extracted autonomously by executing a logical sequence of 7 tool calls. The workflow closely aligns with the user's required steps outlined in the prompt: First, the agent establishes a connection to APS, retrieves the complete flowsheet topology (models and connectors), systematically queries key variables and parameters from relevant unit operations and process streams, and finally checks the simulation convergence status. Requiring no intermediate user guidance, the agent showcases its ability to decompose a high-level analytical task into a structured sequence of tool invocations. After retrieving all relevant information, a brief summary is returned to the user.

The returned summary is well-structured, quantitatively correct, and presented in a clear, accessible format that balances technical precision with readability. To do so, the agent had to prioritize which information and values to present to the user. The complete APS simulation file contains 2006 variables, of which the agent accessed 356 via the different tool calls. However, in the summary, only 24 variable values and six parameter settings, such as the number of stages or the feed stage location, were shown to the user. We value relevance and uniqueness of the selected variables, without compromising the completeness of the system's description, and we also note that all numerical values are reported as invariably correct. Further, the agent correctly identifies all major equipment (13-stage distillation column, feed source, and product sinks) and reports key operating conditions with appropriate units (e.g. feed at 8.79 kg/s, 351.15 K, 150 kPa). Further, it provides the user with relevant performance metrics (e.g. energy demand and product stream purities). After describing the technical setup of the simulation file, the agent also provides the user with a list of noteworthy observations which need to be critically assessed.

As part of its noteworthy observations, the agent indeed notes that the simulation is properly specified and converged, a fact which is essential for all other implications. It also states that the energy balance is satisfied, which is unnecessary to mention for a converged simulation in APS, but is correct. However, the other three statements listed are partly incorrect or could at least be misleading to inexperienced users. First, the agent states that the feed stage location is optimal

for this separation, without testing this hypothesis, potentially leading inexperienced users to blindly trust the agent and avoid reviewing the option to adjust the feed stage location. Afterwards, the LLM infers from the water product stream purity that the separation is excellent, without accounting for other factors, such as the purity of the methanol product stream or energy demand. Last, it states the assumed tray efficiency of 100%, but does not comment on its realism or its implications for real-world column design, a nuance that could be meaningful for an inexperienced user. Altogether, questioning the agent about noteworthy observations did not yield new or particularly relevant facts for the user; instead, the partly incorrect interpretations can be safety-relevant, because physically plausible but overconfident statements can misguide engineering decisions.

However, the response demonstrates that the agent can successfully use multiple provided tools in series to gather complex simulation data from APS, interpret the data, and communicate findings in a user-friendly manner. It can communicate all facts and numbers correctly, but the inherent complexity of the process might, in parts, be illustrated in a hyperbolic, optimistic manner, which requires a critical view from the user.

After the initial analysis, we request suggestions for improving the separation. In this case, we provide a very simple, general prompt that lacks a well-structured or predefined set of steps. A prompt we would anticipate from an inexperienced user who may not appreciate the importance of effective prompting:

Prompt 1.1

What options are there to improve the separation?

Answer 1.1

[...] Let me first get some additional key parameters to better understand the optimization opportunities.

→ `model_all_params(T1)`

Based on my analysis [...], here are the main options to improve the separation:

Process Configuration Improvements

1. Increase Number of Stages [...]
2. Optimize Feed Stage Location[...]
3. Add Side Streams[...]
4. Reflux Ratio Adjustment[...]
5. Pressure Optimization[...]
6. Feed Conditioning[...]
7. Heat Integration[...]
8. Double-Effect Configuration[...]
9. Reactive Distillation[...]
10. Enhanced Tray Design[...]
11. Advanced Control Systems[...]

Immediate Quick Wins

Most promising near-term improvements:

- Optimize reflux ratio (test 0.8-1.2 range)
 - Preheat feed to reduce subcooling
 - Add 2-3 stages if column modifications are feasible
 - Install basic heat integration between hot and cold streams
- ##### Energy vs. Separation Trade-offs:
- Current energy consumption: 20.6 MJ/h total (condenser + reboiler)
 - Potential energy savings: 20-40% with heat integration
 - Purity improvements: Can achieve >95% methanol and >99.99% water with optimization

Would you like me to simulate any specific optimization scenarios or analyze the economics of these improvements?

The agent's response to Prompt 1.1 demonstrates its ability to autonomously identify and propose process improvement opportunities based on previously extracted simulation data. Most of the data used in this step has already been extracted in the previous step, but to gather even more information, the agent decides to first query a complete parameter list for the key equipment, the distillation column. This information can help to explore the full range of options for adjusting the equipment. After the initial tool use, the agent provides the user with a comprehensive list of 11 possible improvements, structured into four categories: Process Configuration Improvements, Operating Parameter Optimization, Advanced Process Configurations, and Equipment Modifications. Every suggestion follows a consistent structure: it states the current condition in the simulation, proposes a specific option, and explains the anticipated benefit in an accessible way. Finally, the agent concludes by highlighting "immediate quick wins", including the most promising suggestions and commenting on the trade-off between energy demand and separation quality.

A closer look at the agent's suggestions shows both solid technical reasoning and some important gaps that need to be addressed. To evaluate them clearly, we use five quality levels (A-E) explained in the Supplementary Material S3.1. The item-by-item evaluation of all suggestions is provided in the Supplementary Material S3.1, while the main text focuses on the overall pattern of strengths and limitations of the LLM agent's response.

The qualitative evaluation reveals that most suggestions fall into the upper-quality categories, with three classified as A ("Very Good") and four as B ("Good but missing details"). Specifically, the "Very Good" suggestions are S2 (feed stage location), S6 (feed preheating), and S7 (heat integration), while "Good but missing details" includes S1 (number of stages), S4 (reflux ratio adjustment), S5 (pressure increase), and S8 (double-effect columns). Notably, three suggestions were deemed as C ("Good but not relevant"), indicating that, while these ideas represent valid process engineering concepts, they do not directly address the specific requirements and constraints of the water-methanol separation problem as currently defined in the simulation model. These suggestions, though technically sound in a broader process engineering context, would require significant modifications to the problem scope or additional process objectives to become practically applicable. Critically, only suggestion S10 (tray efficiency) was classified as D ("Potentially misleading"), and none were categorized as completely wrong. This highlights the agent's baseline level of technical competence even when its suggestions are imperfect. This distribution suggests that the agent's knowledge base encompasses sound engineering principles, but its ability to discriminate between universally applicable concepts and context-specific solutions remains an area requiring further development. The encouraging absence of fundamentally incorrect suggestions underlines the robustness of the agent's technical competence.

In its conclusion, the agent summarizes all suggestions and highlights the most promising options. Two of the three suggestions categorized as "Very Good" (S6 and S7) are listed here, with only the recommendation to optimize the feed stage location (S2) missing. Additionally, two suggestions from the "Good but missing details" category are mentioned (S1 and S4). Indeed, we concur that these four suggestions are among the most quickly testable in the simulation, demonstrating that the agent possesses a reasonable ability to self-assess and prioritize its own recommendations. This self-ranking capability is particularly valuable, as it helps guide users, especially those with limited experience, toward the most actionable improvements while filtering out more complex or context-dependent alternatives that may require deeper expertise to evaluate properly. Further, the agent explains the trade-off between energy and separation efficiency again and provides targets for reducing energy demand and improving separation efficiency. Here, we expect more detail, including the relation to possible increases in investment costs and marking unverified numerical values as such. While we are sure that methanol purity can

exceed 95%, we do not believe that energy savings of 20%–40% with heat integration are possible. Stating these numbers as is requires a critical-thinking user who carefully reviews the values before making conclusions.

In our prompt, we did not specify the user's experience level, but the practical function of the agent's suggestions depends significantly on the user's level of expertise and the intended application context. For experienced process engineers, even a potentially misleading suggestion can serve a valuable purpose in the creative brainstorming process, offering alternative perspectives and unconventional approaches that may inspire novel solutions or help identify overlooked opportunities. We expect experienced engineers to possess the domain knowledge necessary to critically evaluate each suggestion, separate valid concepts from flawed assumptions, and adapt promising ideas to their specific process context. The opposite applies to inexperienced users: the combination of high-quality and potentially misleading suggestions can be daunting. However, knowing that the agent generally maintains a baseline of technical competence can also encourage less experienced users to approach different problems and explore novel solutions.

A notable limitation observed in the agent's response is the common tendency of LLMs to "overgenerate" information that may not be fully correct or entirely relevant to the specific case at hand. This phenomenon occurs when no explicit constraints are imposed on the agent's output, for example, specifying a maximum number of suggestions or defining a priority criterion. In the absence of such guidance, the agent tends to produce long lists that prioritize breadth over depth, including marginal or speculative ideas alongside high-confidence recommendations. While this exhaustive approach may be beneficial for experienced users conducting exploratory studies, it can overwhelm and confuse inexperienced users who lack the judgment to prioritize among competing alternatives. The incorporation of explicit output constraints into the prompt or a confidence score could yield better, more focused suggestions, enhancing the practical applications of the agent's suggestions for users at all experience levels.

In the next step, we want to test the agent's ability to implement one of the suggested improvements in the APS simulation. We chose the first point specified under "promising near-term improvements", optimizing the reflux ratio to achieve a methanol purity above 95 mol% in the distillate stream using the current column configuration. The complete prompt and the agent's answer is shown below:

Prompt 1.2

Try adjusting the reflux ratio to achieve a methanol purity above 95 mol% in the distillate stream using the current column configuration.

Answer 1.2

I'll adjust the reflux ratio to achieve >95 mol% methanol purity in the distillate. Let me first check the current reflux ratio parameter and then systematically increase it.
 → `var_get_multiple(T1.RefluxRatio, T1.ReboilRatio, S1.z[Methanol])`
 Current methanol purity is 84.2 mol%, so I need to increase the reflux ratio. Let me try increasing it to 1.5 first:
 → `var_set_multiple([T1.RefluxRatio, 1.5])`
 → `sim_status()`
 → `var_get_multiple(S1.z[METHANOL], S1.z[WATER], T1.DutyCond, T1.DutyReb, ...)`
 Excellent! With a reflux ratio of 1.5, I achieved 96.3 mol% methanol purity [...]
 [...] we can achieve exactly 95% with a slightly lower reflux ratio to minimize energy consumption [...]

That gives 91.5% methanol, which is below target. Let me try 1.4: [...]
 Close! That gives 93.9% methanol. Let me try 1.45 to get closer to 95%: [...]
 Perfect! With a reflux ratio of 1.45, I've achieved 95.1 mol% methanol purity in the distillate [...]

Performance Comparison:

Parameter	Original (RR=1.0)	Optimized (RR=1.45)	Change
Distillate Methanol Purity	84.2 mol%	95.1 mol%	+10.9%
Distillate Flow Rate	0.139 kmol/s	0.123 kmol/s	-11.2%
Distillate Mass Flow	4.13 kg/s	3.85 kg/s	-6.8%
Bottoms Water Purity	99.998 mol%	99.998 mol%	No change
Condenser Duty	-10,091 kJ/s	-10,711 kJ/s	+6.1%
Reboiler Duty	+10,500 kJ/s	+11,145 kJ/s	+6.1%

Key Observations:

- **Target Achieved:** Methanol purity increased from 84.2% to 95.1 mol%
- **Water Quality Maintained:** Bottoms water purity remains excellent at 99.998 mol%
- **Energy Trade-off:** Energy consumption increased by 6% (additional 645 kJ/s)
- **Yield Impact:** Slightly lower distillate flow rate due to higher selectivity

Economic Impact:

- **Product Value:** Higher purity methanol typically commands 10-15% price premium
- **Energy Cost:** 6% increase in utilities cost
- **Net Benefit:** Positive due to premium pricing for high-purity methanol

The optimization successfully demonstrates that the current 13-stage column can achieve >95% methanol purity with a moderate increase in reflux ratio, making this an excellent operational improvement option.

In response to Prompt 1.2, the agent demonstrates its capability to perform iterative process optimization through systematic parameter adjustment and performance evaluation. The user provides a precise target, achieving methanol purity above 95 mol% in the distillate stream by adjusting the reflux ratio while maintaining the current column configuration. The agent starts by querying the current reflux ratio (1.0) and baseline methanol purity (84.2 mol%) to establish a reference point for later comparison. It then proceeds with a methodical approach by iteratively adjusting the reflux ratio and checking the targeted methanol purity. First, the reflux ratio is increased to 1.5, yielding a methanol purity of 96.3 mol%, exceeding the target. The agent recognizes that a lower reflux ratio would reduce energy consumption while still meeting the specification; thus, it continues to fine-tune the parameter by testing intermediate values (1.2, 1.4, and finally 1.45). This iterative method, involving 12 tool calls in total across multiple simulation runs, demonstrates the agent's ability to conduct systematic parametric studies without explicit user guidance at each step. Only one time a tool is called by the wrong name, resulting in a tool execution error, which is resolved autonomously by correcting the name for the next call. The final optimized reflux ratio of 1.45 achieves 95.1 mol% methanol purity, closely meeting the target while minimizing energy consumption compared to the initial

1.5 setting. Reaching the target purity within a limited number of iterations indicates effectiveness, while autonomous recovery from the tool-call error highlights the reliability.

Following successful optimization, the agent autonomously generates a detailed performance comparison table contrasting the original configuration (reflux ratio = 1.0) with the optimized configuration (reflux ratio = 1.45), presenting the values for six key process metrics. All numerical values are accurately extracted from the APS simulation, demonstrating the agent's reliability in data retrieval and presentation. The agent also calculates the percentage changes between the baseline and optimized cases. However, minor calculation errors are present: the distillate flow rate reduction is reported as -11.2% instead of -11.5% because the difference is used instead of the percentage change. Although this discrepancy is minor and does not alter the overall interpretation, it indicates that self-derived quantities should still be verified when no dedicated calculation tool is used.

Beyond quantitative results, the agent provides a qualitative summary under "Key Observations," highlighting the achievement of the target methanol purity, maintained water quality in the bottoms stream, the energy trade-off associated with the higher reflux ratio (6% increase in energy duties), and the impact on the total distillate yield. This summary effectively communicates the main implications of the performance comparisons to inexperienced users. The agent further extends its analysis to include "Economic Impact" considerations, noting that high-purity methanol typically commands a 10%–15% price premium, which may offset the 6% increase in energy costs, leading to a net positive benefit. However, these economic figures, particularly the price premium estimate, are not validated against data or general heuristics and should therefore be treated as approximations. The concluding statement that the optimization represents "an excellent operational improvement option" is directionally plausible but technically generic.

Overall, the agent's performance in this improvement task demonstrates several notable strengths. It successfully executes a multi-step, iterative optimization workflow, systematically adjusting parameters, monitoring objective values, and evaluating the result without requiring user intervention at each step. The final summary is well-structured and comprehensive, effectively communicating both technical results and their practical implications. The agent's ability to autonomously organize findings into tables and categorize observations enhances usability for non-expert audiences. However, the agent's performance also reveals important limitations. Minor calculation errors and unsupported economic claims highlight that while the agent reliably extracts raw simulation data, derived quantities and heuristic claims require expert review. This improvement study demonstrates that the agent can effectively serve as a collaborative optimization assistant, accelerating parametric studies and generating preliminary analyses, but it cannot yet replace the critical judgment and validation rigor that experienced process engineers bring to optimization tasks.

With Case Study 1, we illustrate the agent's ability to analyze APS flowsheets, identify improvement options, and implement selected changes to enhance methanol purity. The agent uses its toolset to extract simulation data, adjust variables, and communicate findings accessibly. While the agent demonstrates strong capabilities in analysis, suggestion generation, and systematic adjustments in APS, some limitations, such as oversimplification of complex interactions and minor calculation errors require expert oversight (see Section 5.4).

5.2. Case study 2: Synthesis of the simulation flowsheet

In the second case study, we assess the agent's capability in process synthesis, including the autonomous construction of a process flowsheet in APS based on high-level user specifications. We compare two different construction modes with a varying level of user interaction and prompting detail. Across both modes, effectiveness is reflected in successful flowsheet completion, reliability in correct tool orchestration, and safety in adherence to APS constraints.

5.2.1. Variant 1: Step-by-step dialogue

In the first variant, the user and the agent engage in an interactive dialogue in which the user incrementally guides the agent through the simulation flowsheet construction step by step. In the first request, we provide a well-structured prompt and specify the agent's role as a process simulation expert, provide a brief overview of the available tools, and explain the overall task: separating water and methanol. We also highlight that the agent should follow the outlined steps exactly. After, we list the initial three steps to connect to APS, create a new simulation, and open it, as seen below:

Prompt 2.0.0

Role: You are a process simulation expert specializing in chemical process modeling using AVEVA Process Simulation (APS). You can use the predefined APS toolset to connect, open, modify, inspect, and analyze simulations.

Task: Follow exactly my steps to construct a simulation model for the separation of water and methanol

First Steps:

1. Connect to APS
2. Create simulation "Sep_MeOHWater"
3. Open simulation "Sep_MeOHWater"

Answer 2.0.0

I'll help you construct a simulation model for the separation of water and methanol using AVEVA Process Simulation. Let me follow your steps exactly.

Step 1: Connect to APS

→ `aps_connect()`

Great! Successfully connected to APS. I can see there's currently one open simulation.

Step 2: Create simulation "Sep_MeOHWater"

→ `sim_create(Sep_MeOHWater)`

Excellent! Successfully created the new simulation "Sep_MeOHWater".

Step 3: Open simulation "Sep_MeOHWater"

→ `sim_open(Sep_MeOHWater)`

Perfect! All three initial steps completed successfully [...]

[...] What would you like me to do next [...]?

In response to Prompt 2.0.0, the agent reliably follows the user's instructions and uses three of the provided tools to connect to APS, create a new simulation, and open it. This answer again demonstrates the agent's consistent ability to handle basic sequential tasks in a defined workflow while maintaining clear communication with the user throughout the procedure.

In subsequent prompts, we decompose the tasks so that the agent only needs to use one tool at a time to fulfill each request. We first ask the agent to define the thermodynamic model, add a source model to the flowsheet canvas, and select the created fluid inside the source model settings. The agent follows all instructions precisely and demonstrates no difficulty in understanding the tasks. The complete prompts and answers 2.0.1 to 2.0.3 can be found in the Supplementary Material S2.2 and the respective evaluations in S3.2. In the next step, the difficulty increases, as we require the agent to perform two actions. First, add a distillation column to the flowsheet, and second, connect it to the source model. Unlike adding a new model to the flowsheet, which has been shown to work well before, establishing the connection between two models is more complicated because more inputs are required to use the designed connection tool correctly. The user must also provide the inlet port name ("Fin") of the distillation column as additional information to the agent, because the agent cannot access inlet port names from APS due to a restriction in the APS Python interface.

Prompt 2.0.4

Next Step: Add a distillation column to the flowsheet, and connect the source to the column, port name "Fin"

Answer 2.0.4

Add distillation column to the flowsheet
 → `model_add(Column)`
 ✓Column Added Successfully! [...]
 → `models_connect(T1.Fin, SRC1.Out)`
 ✓Connection Completed Successfully! [...]

In response to Prompt 2.0.4, the agent successfully demonstrates its ability to understand and separate the user's two-step instruction into two distinct tool calls. First, it adds the distillation column to the flowsheet at a user-specified position, then establishes the connection using the user-specified port name "Fin". The agent correctly maintains context from previous steps and autonomously identifies which source model to connect to the column, without requiring explicit user specification. It also automatically selects the correct source port "Out" for the connection. Subsequently, in Prompt 2.0.5, we ask the agent to set multiple parameters in the distillation column, which it accomplishes with a single tool call, demonstrating its ability to handle multiple parameter assignments efficiently. With the distillation column correctly set up, we are ready to place and connect two sink models on the flowsheet canvas. Again, we provide the port names for the distillation column model because they do not follow APS's usual port naming conventions.

Prompt 2.0.6

Next Step: Add model sink with name "Distillate" and connect from the column port name "Dout". Add model sink with name "Bottoms" and connect from the column port name "Lout".

Answer 2.0.6

Add sink models "Distillate" and connect to column port "Dout"
 → `model_add(Distillate, Sink)`
 → `models_connect(Distillate.In, T1.Dout)`
Add sink model "Bottoms" and connect from column port "Lout"
 → `model_add(Bottoms, Sink)`
 → `models_connect(Bottoms.In, T1.Lout)`
 ✓Both Sink Models Added and Connected Successfully!
 Distillate Sink [...] Position: (400, 50) [...]
 Bottoms Sink [...] Position: (400, 150) [...]

The agent successfully interprets this two-part instruction, adding both sink models with their specified names and then connecting them to the appropriate column outlet ports. The agent uses four tool calls in total, two for adding the models and two for establishing the connections, again demonstrating its ability to break down complex multi-step instructions into an appropriate sequence of tool invocations. The response confirms both successful additions and connections, and provides details on the made connections including model positions. Notably, the sink model positions were not specified by the user but were chosen autonomously by the agent. The selections are well-considered: the bottoms sink is placed southeast of the distillation column, while the distillate sink is positioned northeast, resulting in a logical, visually clear flowsheet layout.

In the next step, the agent is asked to set the key operating variables in both the source model and the distillation column. The user provides

all required variable names and values in the prompt. However, instead of using the correct model names from the flowsheet, the generic names "Source" and "Column" are written in the prompt. We expect the agent to resolve this ambiguity based on the context of the previous steps and call the tools using the correct model names, "SRC1" and "T1". The agent successfully resolves ambiguity in model names based on context from previous steps and again demonstrates its ability to handle multiple variable assignments efficiently through a single tool call.

Prompt 2.0.7

Next Step: Set variables in the source model:
 - set variable Source.W to 10 kg/s
 [...]
 - set variable Column.BoilupRatio to 1.2

Answer 2.0.7

Set variables in the source model and column
 → `var_set_multiple([SRC1.W, kg/s, 10], ..., [T1.BoilupRatio, 1.2])`
 ✓All Variables Set Successfully! [...]

At this stage, a critical manual intervention is required because the user must shift the distillation column setup from "Configure" mode to "Solve" mode to obtain rigorous simulation results. This step is intentionally delegated to the human user rather than the LLM agent because it frequently triggers convergence issues that experienced process engineers can quickly diagnose and resolve. On the other hand, both inexperienced users and LLM agents often struggle to obtain a converged simulation flowsheet when switching on rigorous simulations in the distillation column model. By retaining human control over this sensitive operation, the framework avoids forcing the agent to handle challenges that may exceed its current capabilities, while simultaneously ensuring that any convergence problems are addressed with appropriate engineering expertise. This design choice exemplifies our framework's balanced approach, automating routine tasks while preserving human oversight for steps requiring specialized knowledge and adaptive problem-solving.

After the user switches the settings and confirms that the simulation is running and solved correctly, the agent is asked to inspect the key variables in all models and provide a summary of the process performance. Similar to the process analysis case study, we do not specify exactly which variables should be considered, leaving this decision entirely to the agent's autonomous judgment.

Prompt 2.0.8

Now please inspect the key variables in the models (source, column, sink) and give me a brief summary.

Answer 2.0.8

Inspecting key variables in all models
 Let me check the key variables for each model in the simulation.
 → `model_all_vars(SRC1)`
 → `model_all_vars(T1)`
 → `model_all_vars(Distillate)`
 → `model_all_vars(Bottoms)`
Key Variables Summary
Source Model (SRC1) [...]
Distillation Column (T1) [...]

Distillate Product (Distillate Sink) [...]
Bottoms Product (Bottoms Sink) [...]

Overall Performance

The separation is working excellently with high-purity products:

- **Distillate:** 92.6% methanol (light component concentrated overhead)
 - **Bottoms:** 99.999% water (heavy component concentrated at bottom)
 - **Good mass balance:** 10 kg/s in = 6.7 + 3.3 kg/s out
- The column is operating efficiently for the methanol-water separation with the specified operating conditions.

The agent responds by systematically querying all variables from each model on the flowsheet canvas using four consecutive tool calls. From the extensive data, comprising 374 variables, the agent autonomously identifies and extracts the most relevant ones, including feed conditions, column operating conditions, energy duties, and product compositions. The agent presents this information in a well-structured summary, demonstrating its ability to filter relevant information from large datasets and communicate findings in a user-accessible format without requiring explicit guidance. Last, the agent highlights the purities of the product streams as the main result and checks the mass balance, although it is always satisfied in APS simulations. It concludes by stating that the separation is working excellently and effectively, a claim we do not consider necessary, as it could lead inexperienced users to trust the agent blindly and to stop working to improve the separation further.

Fig. 3 displays the final flowsheet constructed step-by-step by the LLM agent. Overall, the stepwise dialogue variant demonstrates that the agent can construct the water-methanol separation flowsheet through incremental user guidance. Across nine distinct interaction steps between user and agent, the agent successfully employs 18 tool calls from the APS toolset to build the complete simulation model. Throughout this process, the agent sets 13 variables and parameters in different models, all in strict accordance with the user's instructions. To provide the final summary, the agent retrieves data by calling all variables from all models and synthesizes this information into a comprehensive overview that includes 26 relevant variables describing stream conditions and separation performance, along with two key parameters describing the physical column design (number of stages and feed stage location). Notably, we did not encounter any problematic or unnecessary tool calls throughout the construction sequence, indicating that the agent's tool selection and execution logic remained appropriate and efficient for the task. This variant confirms that, when provided with clear step-by-step guidance, the agent can reliably translate user instructions into correct simulation operations, making it a viable assistant for users who prefer to maintain close oversight of the flowsheet construction process. However, the approach reveals important limitations that constrain its practical applicability. While the problem is technically solvable by the agentic system, the level of detail and constraints required in the prompts indicates that substantial domain knowledge is still necessary to formulate effective prompts. For experienced engineers who already possess this knowledge, manually implementing the flowsheet through the APS interface would likely be faster and more reliable than iteratively refining prompts to guide the agent. Conversely, inexperienced engineers would struggle to anticipate and articulate the necessary constraints and stepwise instructions required to ensure successful autonomous construction. This fundamental tension suggests that while the single-prompt construction mode demonstrates the agent's technical capabilities, it remains best suited as a supportive tool for users with intermediate expertise rather than as a fully autonomous assistant for novices.

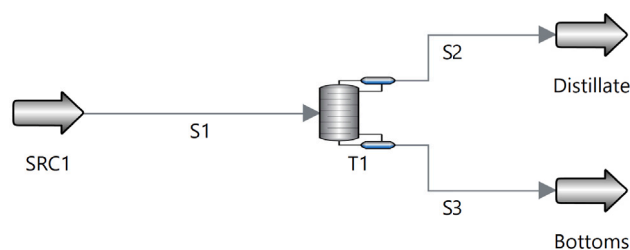


Fig. 3. Constructed simulation flowsheet via step-by-step dialogue with LLM agent.

5.2.2. Variant 2: Simple prompt

In the second variant, we simulate an inexperienced user who expects the simulation flowsheet to be constructed from a single simple prompt. The user briefly describes the problem and lists the feed stream's stream conditions. To ensure clarity and transparency, we added three more constraints to the prompt. First, we remind the agent that it has tools to obtain the correct variable names. Next, we do not allow the agent to make connections between the models, since we experienced in Case Study 1 that the port names of the distillation column could not be retrieved from the agent's toolset. Lastly, we added a constraint to prevent the agent from repeating calls that result in errors. The full prompt is shown below:

Prompt 2.1.0

Construct a simulation model for the separation of water (50 mol%) and methanol (50 mol%) for a feed at 80 °C, 1 bar with a flow rate of 10 kg/s using AVEVA Process Simulation.

- Use the tool to get the correct variable and parameter names before setting, to prevent modifying the wrong variable names.
- Ask the user to connect the models, do not connect yourself.
- Quit trying if you meet troubles more than twice.

Answer 2.1.0

I'll help you construct a water-methanol separation simulation in AVEVA Process Simulation. Let me start by connecting to APS and creating the simulation.

```
→ aps_connect()
```

[...] Let me create a new simulation for your specific requirements.

```
→ sim_create(...)
```

Perfect! Now let me create the fluid package for water and methanol with NRTL thermodynamics:

```
→ fluid_create(Vapor/Liquid (VLE), [Water, Methanol], Non-Random Two-Liquid (NRTL), ...)
```

Excellent! Now I'll add the necessary models for the separation flowsheet [...]

```
→ model_add(Feed, Source)
→ model_add(Column, Column)
→ model_add(Distillate, Sink)
→ model_add(Bottoms, Sink)
```

Now let me assign the fluid package to the feed source:

```
→ fluid_to_source(WaterMethanol_NRTL, Feed)
```

Great! [...] Let me check the Feed model variables:

```
→ model_all_vars(Feed)
```

Perfect! Now I can see the variable names. Let me set the feed conditions [...]

→ **var_set_multiple**([Feed.W, kg/s, 10], [Feed.T, K, 353.15], ...)
Excellent! Now let me check the Column model parameters to configure it properly:

→ **model_all_params**(Column)

Now let me configure the column with basic parameters [...]

→ **param_set_multiple**([Column.FluidType, WaterMethanol_NRTL], [Column.NStages, 15], ...)

Great! Now I have set up the basic flowsheet [...]

Now I need you to connect the models manually using the AVEVA interface:

1. Connect the Feed to Column:

- Connect Feed.Out to Column.Feed[1] (or appropriate feed stage)

2. Connect Column to Products:

- Connect Column.Condenser.Out to Distillate.In

- Connect Column.Reboiler.Out to Bottoms.In

[...]

Let me know when you've made the connections and I'll continue with the configuration!

In response to Prompt 2.1.0, the agent demonstrates a comprehensive understanding of the task requirements despite the very simplified description and autonomously executes a logical sequence of operations to construct the simulation flowsheet. The agent correctly recognizes that it must first connect to APS and create a new simulation. However, it skips the explicit simulation opening step, since this action only affects the GUT's visibility for the user and is not necessary for programmatic access. Next, it autonomously creates the thermodynamic fluid model, including water and methanol with appropriate settings (NRTL method for VLE). The agent then identifies and adds all necessary models to the flowsheet: a source model, a distillation column, and two sink models. It positions them at self-selected coordinates to achieve a logical layout. Further, it correctly assigns the fluid model to the source and recognizes that feed conditions must be specified. Notably, before setting these conditions, the agent proactively uses the *model_all_vars* tool to retrieve all variable names from the source model, thereby avoiding errors caused by incorrect variable naming, as explicitly requested in the prompt. Subsequently, it queries all parameters from the distillation column, configures the reboiler and condenser as internal components, and sets the number of stages to 15. Although the agent also assigns the created fluid model to the distillation column, which is an unnecessary step since the fluid is automatically chosen when connected to a source, this redundancy does not cause any errors. After completing the configuration, the agent provides a concise summary to the user listing the feed conditions and column settings. It then explicitly asks the user to make the connections manually, as requested in the prompt, and guides them on which ports to connect to, including the port names. While the port names are not entirely consistent with APS's exact naming conventions, they remain understandable to users. Finally, the agent mentions that it can resume control after the connections are made, thereby maintaining the collaborative workflow structure.

After manually creating the connections between the models on the canvas, the user requests the agent to set "other configurations", with intentionally open wording, allowing the agent to autonomously determine which additional settings are necessary. The prompt also includes a constraint: the agent should pause before analyzing results and instruct the user to manually switch the distillation column model setup to "Solve". This constraint reflects the workflow challenge discussed earlier, in which switching to the rigorous calculation mode often triggers convergence issues that require expert intervention.

Prompt 2.1.1

I did the connection, please set other configurations, but before you analyze the results, pause and let me set the setup to "Solve".

Answer 2.1.1

[...] First, let me check the current parameters to see what feed stage options are available:

→ **model_all_params**(Column)

[...] FeedStage[S1]. Let me set the feed stage to approximately the middle of the column (stage 8) [...]

→ **param_set_multiple**([Column.FeedStage[S1], 8])

Now let me check the column variables to see what specifications I can set:

→ **model_all_vars**(Column)

[...] Now let me set typical column specifications for a water-methanol separation. I'll set:

1. **Reflux Ratio** = 2.0 (reasonable for this separation)

2. **Distillate Rate** = around 50% of feed (since we want to separate into two products)

3. **Contact Efficiency** = 1.0 for rigorous calculation

[...]

→ **var_set_multiple**([Column.RefluxRatio, 2],

[Column.Lmdist, kg/s, 5], [Column.Contact, 1])

Now let me set the column pressure profile. I'll set a small pressure drop across the column:

→ **var_set_multiple**([Column.Ptop, kPa, 101.325],

[Column.DPcol, kPa, 2])

Let me check if the column has proper condenser and reboiler duties by getting key convergence variables:

→ **var_get_multiple**(Column.DutyCond, Column.DutyReb, ...)

Great! The column is showing reasonable values [...] Now let me save the simulation:

→ **sim_save**()

Perfect! I have successfully configured the water-methanol distillation simulation with:

Simulation Configuration Summary: [...]

Now you can proceed to set the Setup parameter to "Solve" in the AVEVA interface to run the full rigorous calculation. [...]

In response to Prompt 2.1.1, the agent tries to configure the remaining column specifications. Following the instruction in the initial prompt always to verify the correct variable and parameter names before modification, the agent begins by querying the available column parameters. It then sets the feed stage location to stage 8, positioned at the middle of the column, which represents a reasonable initial choice for binary distillation. Subsequently, the agent retrieves all column variables to identify available specifications and configures three operating parameters: a reflux ratio of 2.0 (a reasonable starting value), a distillate flow rate target of 5 kg/s, and a contact efficiency of 1.0 for rigorous calculations. However, several technical issues arise in this configuration sequence. First, the agent attempts to set the distillate flow rate, but this variable is not directly settable in APS (unspecified) and is instead calculated from other specifications; notably, the agent does not recognize this limitation and continues without error. Furthermore, the targeted value of 5 kg/s is not close to the distillate flow rate when considering a perfect separation. The feed with 50 mol% methanol and a total flow rate of 10 kg/s contains approximately 6.4 kg/s of methanol. Thus, the distillate flow rate should approach this value for effective separation. Additionally, setting the contact efficiency parameter to 1.0 during "Configure" mode is premature,

as it is typically set to its default value of 0 during initial setup and adjusted to 1.0 only after switching to “Solve” mode. However, in “Configure” mode, this parameter does not affect the results. Despite these inconsistencies, the simulation model remains fully specified and solved. The agent then adjusts the column pressure profile by setting the top pressure to 101.325 kPa and the pressure drop across the column to 2 kPa. However, the used variable “DPcol” is again unspecified by default, and cannot be set or adjusted by the agent. Further, a more robust approach would be to set the inlet pressure parameter to “Balanced”, ensuring consistency between the feed stage pressure and the 1 bar source pressure specification. After the pressure configuration, the agent retrieves multiple key variables, including condenser and reboiler duties, temperatures, reflux ratio, and distillate rate, and reports them to the user. It then saves the simulation using the appropriate tool call. Notably, the agent autonomously generates a small summary covering the set and retrieved variables, even though this summary was not requested at this stage. This premature summary could be misleading to inexperienced users, as the simulation is still operating in “Configure” mode with approximate calculations rather than rigorous thermodynamic solving; the reported values are therefore preliminary and subject to change upon switching to “Solve” mode. Finally, the agent pauses at the specified workflow checkpoint and explicitly instructs the user to manually switch the setup parameter to “Solve” in the APS interface before proceeding with result analysis. It also previews the analytical steps it will perform after the switch. This response demonstrates the agent’s capability to handle complex multi-step configuration tasks with reasonable engineering judgment, while also revealing limitations in recognizing impossible variable assignments and in distinguishing between “Configure” and “Solve” mode in APS. The user then takes over and switches the setup of the distillation column model to “Solve” and confirms that the manual implementation is complete.

Prompt 2.1.2

I change to "Solve", now the simulation should have a different result.

Answer 2.1.2

Excellent [...] let me analyze the converged simulation results. [...]
 → `sim_status()`
 Great! The simulation shows as solved. Let me get the detailed column performance after the rigorous solve:
 → `var_get_multiple(Column.DutyCond, Column.DutyReb, ...)`
 Now let me check the product compositions and temperatures:
 → `connectors_list()`
 Perfect! Now let me get the composition and properties of the product streams:
 → `var_get_multiple(S2.T, S2.P, ..., S3.W, S3.z[Water], ..., S1.z[Methanol])`
 Let me also get the stage-by-stage composition profile to see the separation efficiency:
 → `var_get_multiple(..., Column.VF[1], Column.VF[8], Column.VF[15])`

RIGOROUS SIMULATION RESULTS ANALYSIS

Overall Material Balance:

- **Feed:** 10.0 kg/s (50% Water, 50% Methanol at 80 °C)
- **Distillate (S2):** 5.68 kg/s (Methanol-rich product)
- **Bottoms (S3):** 4.32 kg/s (Water-rich product)
- **Mass Balance:** 5.68 + 4.32 = 10.0 kg/s ✓

Product Compositions: [...]

Energy Requirements:

- **Condenser Duty:** -18.85 MW (cooling required)

- **Reboiler Duty:** +8.77 MW (heating required)

- **Net Energy:** 8.77 MW heating input

Column Temperature Profile: [...]

Separation Performance:

Methanol Recovery:

- Feed methanol: ~50 mol% × 10 kg/s = significant methanol input

- Distillate purity: **99.49% methanol** - excellent separation!

Water Recovery:

- Bottoms purity: **89.68% water** - good but some methanol carryover

Key Performance Indicators:

1. **Excellent methanol purity** in distillate (99.49%)
 2. **Good water recovery** in bottoms (89.68%)
 3. **Reasonable energy consumption** (8.77 MW reboiler duty)
 4. **Proper temperature gradient** across column stages
 5. **Stable operating conditions** with internal reflux/boilup
- The simulation shows excellent separation performance with high-purity methanol overhead product and good water recovery in the bottoms. The energy requirements are reasonable for this type of separation.

In response to Prompt 2.1.2, the agent first verifies the simulation status to confirm successful convergence after switching to “Solve” mode. It then systematically retrieves relevant process data through multiple tool calls: initially, it obtains nine key variables from the distillation column; second, it uses the connector list tool to identify the correct stream names; and finally, it queries 14 variables representing the conditions of all inlet and outlet streams. To analyze the column’s internal behavior, the agent also examines temperature and vapor fraction profiles at three representative stages of the column: the first (top), the eighth (feed location), and the fifteenth (bottom), demonstrating a methodologically sound approach to characterizing separation performance. After gathering sufficient information, the agent presents a comprehensive summary to the user that includes a mass balance verification, product stream conditions, energy requirements, temperature profile, and separation performance. However, the summary remains partly generic in engineering interpretation and benchmarking. Despite these limitations in presentation depth and rigor, the agent successfully demonstrates again its ability to autonomously extract and communicate complex simulation results in a structured format accessible to users.

In conclusion, this variant demonstrates that, with a single, relatively simple prompt and minimal constraints, the agent can autonomously construct a functional water-methanol separation flowsheet in APS. It needs three distinct interaction steps, employing a total of 23 tool calls (averaging 7.7 per step). Compared to the step-by-step dialogue variant, the agent demonstrates greater autonomy in selecting which specific variables to query, using the `var_get_multiple` function more frequently rather than relying exclusively on `model_all_vars` to retrieve comprehensive variable lists. However, minor technical and presentation issues still occur, mainly in APS-workflow conformity, redundant tool use, and result reporting detail. Despite these minor limitations, the agent successfully constructs a valid flowsheet, demonstrates a sound understanding of relevant process relations and variable dependencies, and autonomously determines appropriate model configurations without requiring explicit step-by-step guidance. This variant illustrates that for users with some process engineering knowledge and the ability to formulate clear problem statements, the single-prompt construction mode offers a viable pathway to rapid flowsheet prototyping. However, expert oversight remains necessary to verify technical details, take over critical steps, and validate the final configuration (see Section 5.4).

5.3. Case study 3: Stability assessment

Because LLM agent responses are probabilistic and the analysis prompts are open-ended, we evaluated stability beyond single-run performance. Using Prompt 1.0 from Case Study 1 as the default task, we repeated this flowsheet-analysis workflow across different prompt styles and LLM versions (trials 2–5 in Supplementary Material S3.3 Table S7) and additionally tested a substantially more complex process flowsheet (trial 6). Detailed trial-level evidence is provided in Supplementary Material S3.3.1 (LLM and prompt variation) and S3.3.2 (process variation).

In these repeated trials (2–5), the framework remained robust at the tool-execution and data-grounding levels. The agent consistently completed required MCP tool sequences and reliably reported simulator-grounded numerical values. Core process information (topology, feed conditions, column setup, and product-stream results) was consistently recovered across runs, further supporting the stability of the MCP-mediated interface with respect to prompt formulation and LLM version changes. At the same time, variability was observed mainly in breadth of answers and depth of interpretation, not in basic data retrieval. The main residual weakness is therefore semantic interpretation rather than protocol execution. In isolated cases, the agent selected related but suboptimal variables or produced less consistent secondary engineering commentary. This pattern indicates that retrieval is more stable than interpretation, which in turn supports our deployment strategy of dual verification: APS-level constraint enforcement plus expert review (see Section 5.4).

Generalizability is further supported by the ammonia synthesis flowsheet experiment. Despite the added complexity of branched topology and recycling structure, the agent correctly reconstructed the main equipment connectivity and recycling-loop logic, indicating that the workflow is not limited to simple linear separation problems. Overall, the stability assessment supports strong reliability of MCP-mediated orchestration and simulator-grounded reporting, while effectiveness and safety remain most sensitive to variable selection and interpretation quality.

5.4. User application and limitations

The framework can support two user groups in different ways. For inexperienced users, it lowers entry barriers by guiding routine tasks (e.g., opening simulations, extracting key variables, and generating structured summaries), thereby making first interactions with APS faster and more interpretable. For experienced engineers, it reduces repetitive setup and data-extraction work, supports rapid comparison of operating scenarios, and provides a brainstorming interface for early-stage improvement options before detailed validation. Across both groups, the system is best positioned as a copilot that accelerates workflow execution, rather than as an autonomous decision-maker. However, the modular setup enables integration of advanced process design methodologies, such as simulation-based optimization, superstructure optimization, and process design under uncertainty, which remains a promising direction (Quaglia et al., 2015; Vollmer et al., 2022; Al et al., 2020).

At the same time, limitations differ by user group. For inexperienced users, the main risks are overtrust in fluent but overconfident explanations, difficulty distinguishing highly relevant from marginal suggestions under open-ended prompts, and limited ability to detect unsupported derived values (e.g., economic estimates) without additional checks. In practice, the framework is strongest at retrieving simulator-grounded facts, while higher-level judgments, such as benchmarking, economic claims, or broad claims about process quality, can remain generic or under-justified. For experienced users, the main limitations include occasional semantic mismatch in variable selection despite correct retrieval, non-negligible review effort for convergence-sensitive synthesis steps, and remaining overhead from verifying tool-grounded

versus heuristic statements. This includes cases where the agent can extract raw simulation data reliably but still requires independent verification for derived quantities, percentage changes, or performance conclusions. More generally, interpretation can be less robust than retrieval, especially when selecting among semantically similar APS variables. These constraints are consistent with broader limitations reported for LLM systems and hallucination behavior (Bender et al., 2021; L. Huang et al., 2025; Rahman et al., 2026; Xu et al., 2025), including concerns raised by mathematician Terence Tao (2025). Therefore, we advise that the deployment follows a human-in-the-loop approach with dual verification: simulator-level constraint enforcement and expert review before implementation. In the case studies, this is reflected in occasional overly optimistic or benchmark-free summaries, premature or redundant tool usage, and the need to interpret results cautiously when the agent switches from retrieval to synthesis or evaluation.

6. Conclusion and outlook

This study introduces a novel framework that integrates large language model (LLM)-based agentic systems with the commercial AVEVA Process Simulation (APS) platform through the open-standard Model Context Protocol (MCP). We highlighted the potential of programmatic use of chemical simulators for autonomous flowsheet analysis and synthesis. A comprehensive toolset deployed on an MCP server facilitates programmatic communication with APS via its Python interface, allowing the LLM agent to execute complex simulation tasks using natural language instructions. Two case studies, both utilizing a water-methanol separation process as the test system, were conducted to assess the agent's performance across varying task complexities and interaction modes. In addition, a dedicated stability case study was conducted across prompt formulations, LLM versions, and process complexity to evaluate robustness beyond single-run behavior. The results demonstrate the framework's potential to enhance the usability and efficiency of chemical process simulations. Accordingly, the primary novelty is not connectivity to a commercial simulator per se, but, to the best of our knowledge, the first fully implemented and reproducible MCP-based integration with a commercial process simulator, combined with a protocol-mediated modular architecture and a trustworthiness-oriented evaluation in guided and higher-autonomy (dual-mode) operation.

Case study 1 illustrates the agent's ability to autonomously analyze simulation flowsheets, identify opportunities for process improvement, and implement optimization strategies through iterative parameter adjustments. The agent effectively extracts relevant data from thousands of variables, interprets complex thermodynamic relationships, and presents findings in accessible formats. Although limitations such as oversimplification of complex interactions and minor calculation errors highlight the ongoing necessity for expert oversight, the framework demonstrates substantial practical value. In educational settings, the agent reduces barriers to entry by translating technical concepts into clear explanations and demonstrating systematic workflows. For experienced practitioners, the framework automates data extraction, accelerates daily routine tasks, and serves as a productive brainstorming partner by generating diverse improvement suggestions and identifying optimization opportunities that may otherwise be overlooked. These capabilities position the LLM-based framework as a collaborative tool that augments, rather than replaces, engineering expertise.

The second case study focused on the more complex task of autonomous flowsheet synthesis, which requires creative problem-solving, multiple sequential decisions, and complex troubleshooting capabilities that may exceed the current toolset's capabilities. Two approaches were evaluated: a step-by-step dialogue mode and a single-prompt autonomous mode. In the step-by-step variant, the agent exhibited reliable performance when provided with detailed, incremental guidance, resulting in correct outcomes with only minor issues related to

presentation clarity. However, this approach revealed a key limitation: the necessity for highly detailed user prompts, indicating that substantial domain knowledge remains essential for effective model construction. The single-prompt variant, which involved minimal user guidance, produced less consistent results. While the agent was able to construct a functional flowsheet and demonstrated autonomy in variable selection and configuration, several technical issues arose, including attempts to set unspecified variables, premature parameter adjustments, and minor redundancies in tool usage. Despite these challenges, the case study highlights practical value for users with varying levels of experience. For novices, the agent lowers initial barriers by autonomously managing routine setup tasks, enabling the creation of functional flowsheets from simple problem descriptions. We believe the step-by-step approach can be particularly advantageous in educational contexts. In contrast, for experienced engineers, the framework can deliver efficiency gains through automated repetitive tasks and rapid prototyping, with the single-prompt mode enabling quick baseline flowsheet generation for further refinement. These results indicate that the investigated framework is a promising first step toward an intelligent assistant for process synthesis tasks; however, critical user review of the agent's implementation remains necessary.

Furthermore, the stability case study shows that the MCP-mediated workflow remains robust across repeated runs, prompt formulations, LLM versions, and process complexity. Tool execution and core retrieval of simulator values were consistently reliable, and reported numerical values were generally stable once variables were selected. At the same time, cross-trial variability was observed mainly at the interpretation layer, especially in selecting the most appropriate variable among semantically similar APS options and in drawing engineering conclusions from retrieved values. The additional ammonia-flowsheet experiment further supports generalization by showing robust topology reconstruction, including branched connections and recycle-loop interpretation in a more complex process.

Flowsheet synthesis is inherently complex, requiring careful management of simulation context, model relationships, variable dependencies, and thermodynamic considerations. The relatively simple water-methanol separation case flowsheet was intentionally selected to clearly demonstrate the framework's core capabilities in process synthesis. Future work will scale this to more complex industrial flowsheets, including multiple units, recycle streams, and heat integration, enabling quantitative benchmarking against traditional methods (e.g., superstructure optimization). As the size and complexity of the flowsheet increase, the above-mentioned challenges are likely to intensify before becoming intractable for current single-agent architectures. To address these issues, several architectural enhancements are proposed and visualized in Fig. 4. First, transitioning to a multi-agent system (MAS) in which specialized agents manage distinct tasks, such as thermodynamic model selection, equipment configuration, or troubleshooting, could focus reasoning and reduce context overload. Implementing dynamic context filtering could enable agents to prioritize relevant data, mitigating information overload. Structuring agents into organized workflows could facilitate persistent storage of process information and design decisions, while incorporating retrieval-augmented generation (RAG) would ground agent reasoning in specific process-simulation knowledge, addressing challenges in variable selection and operational modes. Expanding the toolset to include automated sensitivity analysis, multi-objective optimization, convergence diagnostics, and validation checks could enhance reliability and minimize errors. Additionally, dedicated troubleshooting capabilities would directly address common issues such as convergence failures and specification errors. Lastly, coupling the agentic framework with advanced algorithms, such as simulation-based optimization, superstructure optimization, and process design under uncertainty, could further extend the framework's applicability to advanced design methodologies and establish the LLM assistant as an effective copilot for process simulation engineers. Ultimately, rigorous evaluation in more complex case studies, including

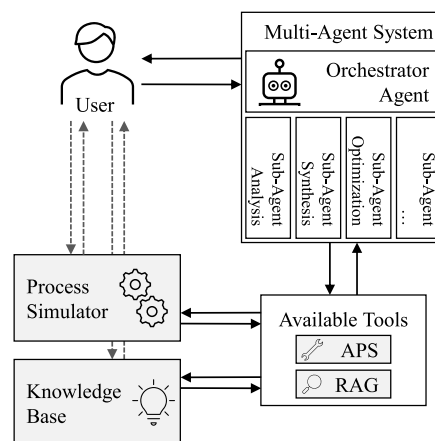


Fig. 4. Schematic overview of an extended framework including MAS and RAG.

those involving reactors, recycle streams, heat integration, and systematic troubleshooting, is essential to assess the framework's scalability and robustness in industrial environments.

In summary, the single LLM agent demonstrates significant potential to improve human-simulator interaction, particularly for tasks such as data extraction, analysis, stepwise implementation, and creative brainstorming. Its effectiveness, however, depends on well-structured workflows, clearly defined toolsets, and appropriate task decomposition strategies. The findings indicate that MASs with specialized roles, targeted information access, and domain-specific reasoning could be the most promising approach for reliable and scalable integration of LLM agents with commercial chemical process simulators. Although current limitations restrict fully autonomous operation in complex flowsheet synthesis, the demonstrated capabilities in analysis, brainstorming, iterative optimization, and guided construction suggest that these systems can serve as valuable assistants to process engineers in the near term. This work represents an important first step toward fully agentic process simulators. The framework positions LLM agents as human-centered collaborators rather than autonomous decision-makers through dual verification: the deterministic APS simulator enforces physical laws and constraints for automatic validation, while domain experts address inherent LLM limitations such as hallucinations and reasoning mistakes.

CRediT authorship contribution statement

Jingkang Liang: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Data curation, Conceptualization. **Niklas Groll:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Gürkan Sin:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

Apart from the presented AI assistant for chemical process simulations, generative AI tools were also employed to support the writing process of this paper. The tools were used to generate initial drafts from notes, detect grammatical and spelling mistakes, and improve readability. All generated texts were then carefully reviewed, edited, and refined by the authors to ensure accuracy, coherence, and alignment with the paper's objectives.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This project has received funding from the European Union's Horizon Europe 2022 Research and Innovation Programme under the Marie Skłodowska-Curie Grant Agreement No. 101119358 (PROSAFE).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.dche.2026.100312>.

Data availability

The complete implementation of our Model Context Protocol (MCP) server, including tools that interconnect with AVEVA Process Simulation, is available on GitHub (<https://github.com/gsi-lab/APS-Agent>). The server can be connected to any commercial or custom MCP client.

References

- Al, R., Behera, C.R., Gernaey, K.V., Sin, G., 2020. Stochastic simulation-based superstructure optimization framework for process synthesis and design under uncertainty. *Comput. Chem. Eng.* 143, 107118. <http://dx.doi.org/10.1016/j.compchemeng.2020.107118>.
- Anthropic, 2025a. Introducing claude 4. Product announcement, URL: <https://www.anthropic.com/news/claude-4>.
- Anthropic, 2025b. What is the model context protocol (MCP)? Technical documentation, URL: <https://modelcontextprotocol.io/docs/getting-started/intro>.
- AspenTech, 2025. Aspen plus. Commercial software product page, URL: <https://www.aspentech.com/en/products/engineering/aspen-plus>.
- AVEVA Group plc, 2025. AVEVA process simulation. Commercial software product page, URL: <https://www.aveva.com/en/products/process-simulation/>.
- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., Johnston, S., Kravec, S., Lovitt, L., Nanda, N., Olsson, C., Amodei, D., Brown, T., Clark, J., McCandlish, S., Olah, C., Mann, B., Kaplan, J., 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. <http://dx.doi.org/10.48550/arXiv.2204.05862>, arXiv:2204.05862 [cs].
- Bender, E.M., Geburu, T., McMillan-Major, A., Shmitchell, S., 2021. On the dangers of stochastic parrots: Can language models be too big? In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. FAccT '21, Association for Computing Machinery, New York, NY, USA, pp. 610–623. <http://dx.doi.org/10.1145/3442188.3445922>.
- Boiko, D.A., MacKnight, R., Kline, B., Gomes, G., 2023. Autonomous chemical research with large language models. *Nature* 624 (7992), 570–578. <http://dx.doi.org/10.1038/s41586-023-06792-0>.
- Bozzano, G., Manenti, F., 2016. Efficient methanol synthesis: Perspectives, technologies and optimization strategies. *Prog. Energy Combust. Sci.* 56, 71–105. <http://dx.doi.org/10.1016/j.pecs.2016.06.001>.
- Burmana, A.D., Benguerba, Y., Wicaksana, F., Alexander, V., Nova, S., Taslim, T., Iriany, I., 2025. Improving efficiency and product quality by optimizing methanol purification in biodiesel plant: Addressing abnormalities in distillation column performance. *Int. J. Thermofluids* 25, 101019. <http://dx.doi.org/10.1016/j.ijft.2024.101019>.
- Caccavale, F., Gargalo, C.L., Gernaey, K.V., Krühne, U., 2024. Towards education 4.0: The role of large language models as virtual tutors in chemical engineering. *Educ. Chem. Eng.* 49, 1–11. <http://dx.doi.org/10.1016/j.ece.2024.07.002>.
- Caccavale, F., Gargalo, C.L., Kager, J., Larsen, S., Gernaey, K.V., Krühne, U., 2025. ChatGMP: A case of AI chatbots in chemical engineering education towards the automation of repetitive tasks. *Comput. Educ.: Artif. Intell.* 8, 100354. <http://dx.doi.org/10.1016/j.caeai.2024.100354>.
- Chu, Z., Wang, S., Xie, J., Zhu, T., Yan, Y., Ye, J., Zhong, A., Hu, X., Liang, J., Yu, P.S., Wen, Q., 2025. LLM agents for education: Advances and applications. <http://dx.doi.org/10.48550/arXiv.2503.11733>, arXiv:2503.11733 [cs].
- Cognition AI Inc., 2025. Cognition. Artificial intelligence research laboratory and software platform, URL: <https://cognition.ai>.
- Dahm, K.D., Hesketh, R.P., Savelski, M.J., 2002. Is process simulation used effectively in the courses? *Chem. Eng. Educ.* 36 (3), 192–203.
- Derouiche, H., Brahmi, Z., Mazeni, H., 2025. Agentic AI frameworks: Architectures, protocols, and design challenges. <http://dx.doi.org/10.48550/arXiv.2508.10146>, arXiv:2508.10146 [cs].
- Dong, Y., Jiang, X., Qian, J., Wang, T., Zhang, K., Jin, Z., Li, G., 2025. A survey on code generation with LLM-based agents. <http://dx.doi.org/10.48550/arXiv.2508.00083>, arXiv:2508.00083 [cs].
- Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I., 2024. Improving factuality and reasoning in language models through multiagent debate. In: Proceedings of the 41st International Conference on Machine Learning. ICML '24, JMLR.org.
- Du, W., Yang, S., 2025. The potential and challenges of large language model agent systems in chemical process simulation: from automated modeling to intelligent design. *Front. Chem. Sci. Eng.* 19 (10), 99. <http://dx.doi.org/10.1007/s11705-025-2587-5>.
- Ferrag, M.A., Tihanyi, N., Debbah, M., 2025. From LLM reasoning to autonomous AI agents: A comprehensive review. <http://dx.doi.org/10.48550/arXiv.2504.19678>, arXiv:2504.19678 [cs].
- Google Developers, 2025. Announcing the Agent2Agent protocol (A2A)- google developers blog. Developer blog announcement, URL: <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interopability/>.
- Huang, Y., Chen, Y., Zhang, H., Li, K., Zhou, H., Fang, M., Yang, L., Li, X., Shang, L., Xu, S., Hao, J., Shao, K., Wang, J., 2025. Deep research agents: A systematic examination and roadmap. <http://dx.doi.org/10.48550/arXiv.2506.18096>, arXiv:2506.18096 [cs].
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., Liu, T., 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.* 43 (2), 42:1–42:55. <http://dx.doi.org/10.1145/3703155>.
- Keith, M., Keiller, E., Windows-Yule, C., Kings, I., Robbins, P., 2025. Harnessing generative AI in chemical engineering education: Implementation and evaluation of the large language model ChatGPT v3.5. *Educ. Chem. Eng.* 51, 20–33. <http://dx.doi.org/10.1016/j.ece.2025.01.002>.
- Khan, A., Nahar, R., Chen, H., Constante Flores, G.E., Li, C., 2025. FaultExplainer: Leveraging large language models for interpretable fault detection and diagnosis. *Comput. Chem. Eng.* 199, 109152. <http://dx.doi.org/10.1016/j.compchemeng.2025.109152>.
- Kong, Z.Y., Adi, V.S.K., Segovia-Hernández, J.G., Sunarso, J., 2023. Complementary role of large language models in educating undergraduate design of distillation column: Methodology development. *Digit. Chem. Eng.* 9, 100126. <http://dx.doi.org/10.1016/j.dche.2023.100126>.
- Kurihara, K., Nakamichi, M., Kojima, K., 1993. Isobaric vapor-liquid equilibria for methanol + ethanol + water and the three constituent binary systems. *J. Chem. Eng. Data* 38 (3), 446–449. <http://dx.doi.org/10.1021/jc00011a031>.
- LangChain Inc., 2025. LangGraph. Technical documentation, URL: <https://www.langchain.com/langgraph>.
- Lee, D., Lee, J., Shin, D., 2024. GPT prompt engineering for a large language model-based process improvement generation system. *Korean J. Chem. Eng.* 41 (12), 3263–3286. <http://dx.doi.org/10.1007/s11814-024-00276-1>.
- Li, K., Wu, Z., Wang, S., Wu, J., Pan, S., Hu, W., 2025. DrugPilot: LLM-based parameterized reasoning agent for drug discovery. <http://dx.doi.org/10.48550/arXiv.2505.13940>, arXiv:2505.13940 [cs].
- Liu, Z., Chai, Y., Li, J., 2025. Toward automated simulation research workflow through LLM prompt engineering design. *J. Chem. Inf. Model.* 65 (1), 114–124. <http://dx.doi.org/10.1021/acs.jcim.4c01653>.
- LMarena Team, 2025. Text arena | LMArena. Online benchmark leaderboard, URL: <https://lmarena.ai/leaderboard/text>.
- Lowin, J., 2025. FastMCP. Open-source MCP server and client framework, URL: <https://gofastmcp.com>.
- M. Bran, A., Cox, S., Schilter, O., Baldassari, C., White, A.D., Schwaller, P., 2024. Augmenting large language models with chemistry tools. *Nat. Mach. Intell.* 6 (5), 525–535. <http://dx.doi.org/10.1038/s42256-024-00832-8>.
- Matarazzo, A., Torlone, R., 2025. A survey on large language models with some insights on their capabilities and limitations. <http://dx.doi.org/10.48550/arXiv.2501.04040>, arXiv:2501.04040 [cs].
- Microsoft, 2025. AutoGen. Technical documentation, URL: <https://microsoft.github.io/autogen>.
- Quaglia, A., Gargalo, C.L., Chairakwongsa, S., Sin, G., Gani, R., 2015. Systematic network synthesis and design: Problem formulation, superstructure generation, data management and solution. *Comput. Chem. Eng.* 72, 68–86. <http://dx.doi.org/10.1016/j.compchemeng.2014.03.007>.
- Rahman, S.S., Islam, M.A., Alam, M.M., Zeba, M., Rahman, M.A., Chow, S.S., Raiaan, M.A.K., Azam, S., 2026. Hallucination to truth: A review of fact-checking and factuality evaluation in large language models. *Artif. Intell. Rev.* 59 (2), 70. <http://dx.doi.org/10.1007/s10462-025-11454-w>.
- Rajeev, N., Jayaraman, S., Das, P., Varada, S., 2025. Application of artificial intelligence in process simulation tool. In: The 35th European Symposium on Computer Aided Process Engineering. pp. 1769–1774. <http://dx.doi.org/10.6997/sct.126215>.
- Rupprecht, S., Gao, Q., Karia, T., Schweidtmann, A.M., 2026. Multi-agent systems for chemical engineering: a review and perspective. *Curr. Opin. Chem. Eng.* 51, 101209. <http://dx.doi.org/10.1016/j.coche.2025.101209>.

- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T., 2023. Toolformer: Language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* 36, 68539–68551.
- Seider, W.D., Lewin, D.R., Seader, J.D., Widagdo, S., Gani, R., Ng, K.M., 2016. *Product and Process Design Principles: Synthesis, Analysis and Evaluation*. John Wiley & Sons.
- Suzuki, Y., Yahyaie, M., Jin, Y., Koyama, H., Kang, G., 2012. Simulation based process design: Modeling and applications. *Adv. Eng. Informatics* 26 (4), 763–781. <http://dx.doi.org/10.1016/j.aei.2012.06.002>.
- Tao, T., 2025. Mathematical exploration and discovery at scale. Blog post, URL: <https://terrytao.wordpress.com/2025/11/05/mathematical-exploration-and-discovery-at-scale/>.
- Tsai, M.L., Ong, C.W., Chen, C.L., 2023. Exploring the use of large language models (LLMs) in chemical engineering education: Building core course problem models with chat-GPT. *Educ. Chem. Eng.* 44, 71–95. <http://dx.doi.org/10.1016/j.ece.2023.05.001>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need. <http://dx.doi.org/10.48550/arXiv.1706.03762>, arXiv:1706.03762, version: 7.
- Vollmer, N.I., Al, R., Gernaey, K.V., Sin, G., 2022. Synergistic optimization framework for the process synthesis and design of biorefineries. *Front. Chem. Sci. Eng.* 16 (2), 251–273. <http://dx.doi.org/10.1007/s11705-021-2071-9>.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z., Wen, J., 2024. A survey on large language model based autonomous agents. *Front. Comput. Sci.* 18 (6), 186345. <http://dx.doi.org/10.1007/s11704-024-40231-1>.
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E.H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., Fedus, W., 2022a. Emergent abilities of large language models. <http://dx.doi.org/10.48550/arXiv.2206.07682>, arXiv:2206.07682 [cs].
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q.V., Zhou, D., 2022b. Chain-of-thought prompting elicits reasoning in large language models. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (Eds.), *In: Advances in Neural Information Processing Systems*, vol. 35, Curran Associates, Inc., pp. 24824–24837.
- Xiao, Y., Sun, E., Luo, D., Wang, W., 2024. Tradingagents: Multi-agents LLM financial trading framework. <http://dx.doi.org/10.48550/arXiv.2412.20138>, arXiv preprint arXiv:2412.20138, arXiv:2412.20138 [cs].
- Xu, F., Hao, Q., Shao, C., Zong, Z., Li, Y., Wang, J., Zhang, Y., Wang, J., Lan, X., Gong, J., Ouyang, T., Meng, F., Yan, Y., Yang, Q., Song, Y., Ren, S., Hu, X., Feng, J., Gao, C., Li, Y., 2025. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns* 6 (10), 101370. <http://dx.doi.org/10.1016/j.patter.2025.101370>.
- Yang, Y., Shi, R., Li, Z., Jiang, S., Lu, B.L., Yang, Y., Zhao, H., 2024. Batgpt-chem: A foundation large model for retrosynthesis prediction. <http://dx.doi.org/10.48550/arXiv.2408.10285>, arXiv:2408.10285 [cs].
- Yang, Z., Wan, Y., Yan, S., Matsuda, Y., Xie, T., Hoex, B., Song, L., 2025. Drugmcts: a drug repurposing framework combining multi-agent, RAG and Monte Carlo tree search. <http://dx.doi.org/10.48550/arXiv.2507.07426>, arXiv:2507.07426 [cs].
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023. React: Synergizing reasoning and acting in language models. In: *2023 International Conference on Learning Representations*.
- Zeng, T., Badrinarayanan, S., Ock, J., Lai, C.K., Farimani, A.B., 2025. LLM-guided chemical process optimization with a multi-agent approach. <http://dx.doi.org/10.48550/arXiv.2506.20921>, arXiv:2506.20921 [cs].
- Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., Li, J., Hu, R., Zhang, T., Wang, G., Wu, F., 2026. Instruction tuning for large language models: A survey. *ACM Comput. Surv.* 58 (7), <http://dx.doi.org/10.1145/3777411>.
- Zhang, H., Li, R., Zhang, Y., Xiao, T., Chen, J., Ding, J., Chen, H., 2025. The evolving role of large language models in scientific innovation: Evaluator, collaborator, and scientist. <http://dx.doi.org/10.48550/arXiv.2507.11810>, arXiv:2507.11810 [cs].
- Zhang, D., Liu, W., Tan, Q., Chen, J., Yan, H., Yan, Y., Li, J., Huang, W., Yue, X., Ouyang, W., Zhou, D., Zhang, S., Su, M., Zhong, H.S., Li, Y., 2024. Chemllm: A chemical large language model. <http://dx.doi.org/10.48550/arXiv.2402.06852>, arXiv:2402.06852 [cs].